JavaScript & the DOM

selecting · traversing · changing · events

1 What is JavaScript?

The three work as a **team** to build a web page:

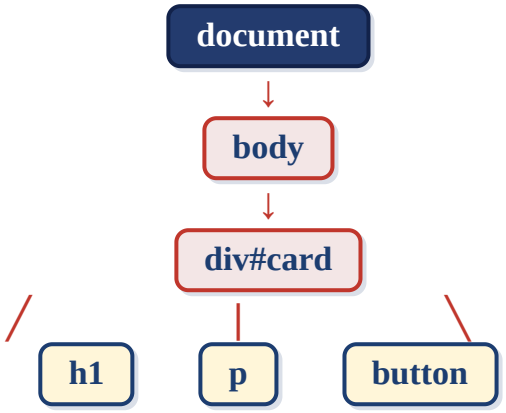
- **HTML** → the structure (skeleton)
- **CSS** → the looks (clothes)
- **JavaScript** → the behaviour (the brain)

> **Remember:** HTML builds it, CSS styles it, JS makes it do things. Today the page stops being a poster and starts responding to people.

2 What is the DOM?

DOM = Document Object Model. When the browser loads your HTML it turns it into a tree of objects. Every tag becomes a **node** your JS can read & change.

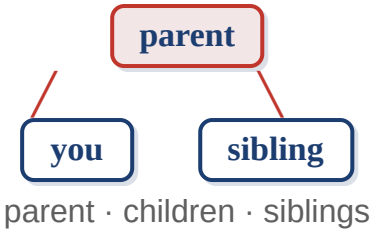
> **Key idea:** the DOM is an API the browser gives you. It is not part of the JavaScript language itself. The browser builds the tree and hands you methods (**document.querySelector**, **.appendChild** ...) to read and change the live page.

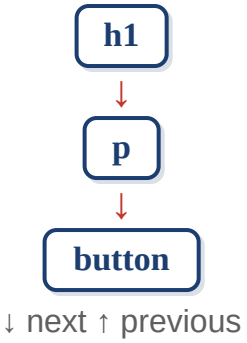


every HTML element = one node (object) in this tree

3 Nodes = a family tree

Every node has a **parent**, **children** and **siblings** - just like a family.





4 Selecting elements (the DOM API)

Before you can change anything, you have to grab it. The browser gives you these finder methods:

```
// by id  -> one element
document.getElementById("card")

// by class name -> HTMLCollection
document.getElementsByClassName("card")

// by tag name -> HTMLCollection
document.getElementsByTagName("li")

// first match of a CSS selector
document.querySelector(".card")

// ALL matches -> NodeList
document.querySelectorAll(".card")
```

Note: querySelector uses the exact same selectors you learned in CSS - so you already know most of this! The **getElementsBy...** ones are older; **querySelector(All)** is the modern go-to.

5 CSS selectors you can use

selector	meaning	example
*	everything (universal)	"*"
#id	by id	"#header"
.class	by class	".card"
tag	by tag name	"button"
.a.b	has BOTH classes	".card.active"
A, B	A or B (group)	"h1, h2"
A B	descendant (any depth)	".card button"
A > B	direct child only	".card > button"
A + B	next sibling right after A	"h2 + p"
A ~ B	any sibling after A	"h2 ~ p"
:first-child	first in its parent	"li:first-child"
:last-child	last in its parent	"li:last-child"
:nth-child(n)	the nth, counts from 1	"li:nth-child(2)"
:nth-child(even)	even ones (2,4,6...)	"tr:nth-child(even)"
:nth-child(odd)	odd ones (1,3,5...)	"tr:nth-child(odd)"
:nth-child(3n)	every 3rd (a formula)	"li:nth-child(3n)"
:nth-of-type(n)	nth of that tag only	"p:nth-of-type(1)"
:only-child	the sole child	"span:only-child"
:not(x)	everything except x	"li:not(.done)"
:hover / :focus	state based	"button:hover"
:checked / :disabled	form states	"input:checked"
[attr]	has the attribute	"a[target]"
[attr="x"]	attribute equals x	input[type="password"]
[attr^="x"]	attribute starts with x	a[href^="https"]
[attr$="x"]	attribute ends with x	img[src$=".png"]
[attr*="x"]	attribute contains x	a[href*="youtube"]

even / odd: nth-child(even) and nth-child(odd) are the classic way to stripe table rows. **3n**, **2n+1** etc. are formulas - n counts 0,1,2,3...

6 Traversing the tree

Once you hold one element, you can walk around it:

- → **.parentElement** - go **up** to the parent
- → **.children** - all direct children
- → **.firstElementChild / .lastElementChild**
- → **.nextElementSibling / .previousElementSibling** - the neighbours
- → **.closest(".card")** - walk **UP** till a match
- → **card.querySelector("button")** - search inside that element only

7 Reading & changing an element

The useful part. Once you have an element, change it live:

```
// text  (textContent = raw text)
button.textContent = "Submit";
button.innerText   = "Submit";
// html inside the element
div.innerHTML = "<h1>Hi</h1>";
// inline style
button.style.color = "red";
button.style.display = "none";
// classes  (let CSS hold the look!)
el.classList.add("active");
el.classList.remove("active");
el.classList.toggle("active");
el.classList.contains("active"); // T/F
// attributes
img.setAttribute("src", "dog.png");
img.getAttribute("src");
img.removeAttribute("alt");
// data-* attrs + form value
el.dataset.userId;   input.value;
```

Rule: prefer classList.toggle over setting styles one-by-one. Keep the **logic in JS**, the **look in CSS.** And **textContent** is safer than **innerHTML** for plain text.

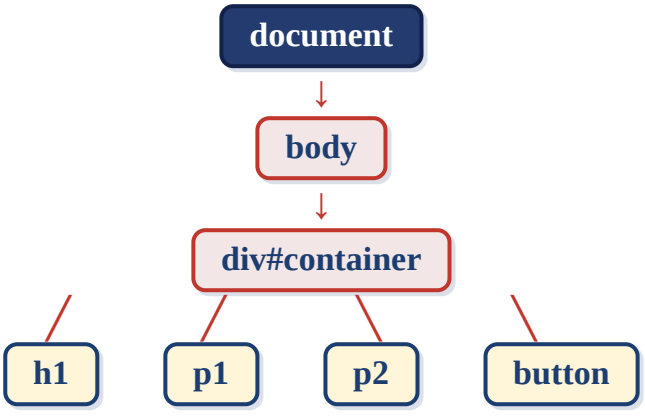
8 Creating, adding & removing

```
const li = document.createElement("li");
li.textContent = "Apple";
// add it to the page
ul.appendChild(li);   ul.prepend(li);
node.before(li);     node.after(li);
ul.insertAdjacentHTML("beforeend", "<li>Hi</li>");
// copy / replace / remove
li.cloneNode(true);   old.replaceWith(li);
button.remove();       // take off page
```

9 HTMLCollection vs NodeList

- **.children** → an **HTMLCollection** - it's live (updates if the DOM changes)
- **querySelectorAll()** → a **NodeList** - a static snapshot you loop with **forEach()**

★ one picture to remember it all



- **parentElement** ↑ up
- **children** ↓ down
- **first/lastElementChild**
- **next/previousElementSibling**
- **closest()** ↑ walks up to match
- **querySelector(All)** = find

Interview rule: only 2 jobs → (1) FIND an element, then (2) move / read / change it. Everything today fits in those two buckets.

★ cheat sheet - the DOM API

method / property	what it does
getElementById()	find by id
getElementsByClassName / TagName()	find by class / tag (older)
querySelector() / querySelectorAll()	find by CSS selector (modern)
parentElement / children	move up / down
first / lastElementChild	first / last child
next / prevElementSibling	the neighbours
closest()	nearest matching ancestor
textContent / innerText / innerHTML	read / change content
style	change inline CSS
classList.add/remove/toggle/contains	manage classes
get / set / removeAttribute()	manage attributes
dataset / value	data-* attrs / form value
createElement()	make a new element
appendChild / prepend / before / after	add to the page
insertAdjacentHTML()	quick html insert
cloneNode() / replaceWith() / remove()	copy / replace / delete